

Problem A. Destruction

Time Limit 2000 ms

Problem Statement

We have a connected undirected graph with N vertices and M edges.

The vertices are numbered 1 through N , and the edges are numbered 1 through M . Edge i connects Vertices A_i and B_i .

Takahashi is going to remove zero or more edges from this graph.

When removing Edge i , a reward of C_i is given if $C_i \geq 0$, and a fine of $|C_i|$ is incurred if $C_i < 0$.

Find the maximum total reward that Takahashi can get when the graph must be connected after removing edges.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $N - 1 \leq M \leq 2 \times 10^5$
- $1 \leq A_i, B_i \leq N$
- $-10^9 \leq C_i \leq 10^9$
- The given graph is connected.
- All values in input are integers.

Input

Input is given from Standard Input in the following format:

```
N M
A1 B1 C1
A2 B2 C2
⋮
AM BM CM
```

Output

Print the answer.

Sample 1

Input	Output
4 5 1 2 1 1 3 1 1 4 1 3 2 2 4 2 2	4

Removing Edges 4 and 5 yields a total reward of 4. You cannot get any more, so the answer is 4.

Sample 2

Input	Output
3 3 1 2 1 2 3 0 3 1 -1	1

There may be edges that give a negative reward when removed.

Sample 3

Input	Output
2 3 1 2 -1 1 2 2 1 1 3	5

There may be multi-edges and self-loops.

In order to prepare the “The First National ACM School Contest” (in 20??) the major of the city decided to provide all the schools with a reliable source of power. (The major is really afraid of blackoutsJ). So, in order to do that, power station “Future” and one school (doesn’t matter which one) must be connected; in addition, some schools must be connected as well.

You may assume that a school has a reliable source of power if it’s connected directly to “Future”, or to any other school that has a reliable source of power. You are given the cost of connection between some schools. The major has decided to pick out two the cheapest connection plans – the cost of the connection is equal to the sum of the connections between the schools. Your task is to help the major — find the cost of the two cheapest connection plans.

Input

The Input starts with the number of test cases, T ($1 < T < 15$) on a line. Then T test cases follow. The first line of every test case contains two numbers, which are separated by a space, N ($3 < N < 100$) the number of schools in the city, and M the number of possible connections among them. Next M lines contain three numbers A_i , B_i , C_i , where C_i is the cost of the connection ($1 < C_i < 300$) between schools A_i and B_i . The schools are numbered with integers in the range 1 to N .

Output

For every test case print only one line of output. This line should contain two numbers separated by a single space – the cost of two the cheapest connection plans. Let S_1 be the cheapest cost and S_2 the next cheapest cost. It’s important, that $S_1 = S_2$ if and only if there are two cheapest plans, otherwise $S_1 < S_2$. You can assume that it is always possible to find the costs S_1 and S_2 .

Sample Input

```
2
5 8
1 3 75
3 4 51
2 4 19
3 2 95
2 5 42
5 4 31
1 2 9
3 5 66
9 14
1 2 4
1 8 8
2 8 11
3 2 8
8 9 7
8 7 1
7 9 6
9 3 2
3 4 7
3 6 4
7 6 2
4 6 14
4 5 9
5 6 10
```

Sample Output

```
110 121
37 37
```

Problem C. Blocks on Grid

Time Limit 2000 ms

Problem Statement

We have a grid with H horizontal rows and W vertical columns. The square at the i -th row from the top and j -th column from the left has $A_{i,j}$ blocks stacked on it.

At least how many blocks must be removed to make all squares have the same number of blocks?

Constraints

- $1 \leq H, W \leq 100$
- $0 \leq A_{i,j} \leq 100$

Input

Input is given from Standard Input in the following format:

```
H W
A1,1 A1,2 ... A1,W
⋮
AH,1 AH,2 ... AH,W
```

Output

Print the minimum number of blocks that must be removed.

Sample 1

Input	Output
2 3 2 2 3 3 2 2	2

Removing 1 block from the top-right square and 1 from the bottom-left square makes all squares have 2 blocks.

Sample 2

Input	Output
3 3 99 99 99 99 0 99 99 99 99	792

Sample 3

Input	Output
3 2 4 4 4 4 4 4	0

Problem D. NewFolder(1)

Time Limit 2000 ms

Problem Statement

For two strings A and B , let $A + B$ denote the concatenation of A and B in this order.

You are given N strings $S_1, \dots, S_N$. Modify and print them as follows, in the order $i = 1, \dots, N$:

- if none of $S_1, \dots, S_{i-1}$ is equal to S_i , print S_i ;
- if X ($X > 0$) of $S_1, \dots, S_{i-1}$ are equal to S_i , print $S_i + (+ X +)$, treating X as a string.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- S_i is a string of length between 1 and 10 (inclusive) consisting of lowercase English letters.

Input

Input is given from Standard Input in the following format:

```
N
S1
S2
⋮
SN
```

Output

Print N lines as specified in the Problem Statement.

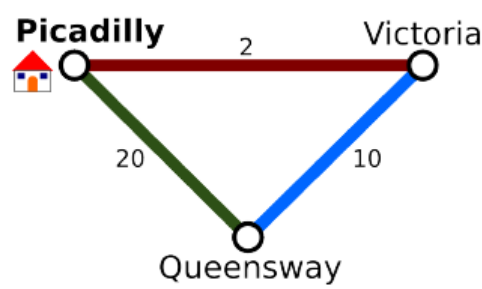
Sample 1

Input	Output
5 newfile newfile newfolder newfile newfolder	newfile newfile(1) newfolder newfile(2) newfolder(1)

Sample 2

Input	Output
11 a a a a a a a a a a a	a a(1) a(2) a(3) a(4) a(5) a(6) a(7) a(8) a(9) a(10)

Peter lives in *Expensive City*, one of the most expensive cities in the world. Peter has not got enough money to buy a car, and the buses in *Expensive City* are pretty bad, so he uses the subway to go to work. Up to now, the subway was very cheap: you could travel anywhere with just one \$2 ticket. Last month, the managers decided that it was too cheap so they invented the EFS (Expensive Fare System). With this system, users can only buy monthly tickets between adjacent stations, which allows them to move between these stations any number of times. The price of the monthly ticket varies between stations, so the decision of which tickets to buy must be taken carefully.



With the previous subway plan, the cheapest way to travel from Picadilly to Victoria and Queensway was to buy the monthly ticket Picadilly-Victoria and Queensway-Victoria, for a total cost of \$12.

Peter is a salesperson, so he needs to be able to travel to any part of the city. He wants to spend as little money as possible, and here is where you come into the picture. He has hired you to write a program that, given the list of stations, the fares of the monthly tickets between pairs of stations and the station nearest Peter’s home, returns the minimum amount of money Peter has to spend in order to travel to any other station. This program also has to return value if it is not possible to go from Peter’s home station to all the rest, because in this case Peter will begin to consider using buses...

Input

The input consists of several test cases. A test case begins with a line containing two integers: $1 \leq s \leq 400$ (the number of stations) and $0 \leq c \leq 79800$ (the number of connections) separated by a single space. This is followed by s lines, each one containing the name of a subway station. These names will be strings of characters (uppercase or lowercase) without punctuation marks or whitespace characters, and with a maximum length of 10 characters. After the names of the stations there will be c lines showing the connections between stations. A connection allows people to travel from one station to the other in both directions. Each connection is represented as two strings indicating the names of the stations and a positive integer indicating the cost of the monthly ticket, all of which are separated by single spaces. All names of stations appearing in the connections will have previously appeared in the list of s stations. The connections will all be different, and there will not be any connection from a station to itself. The test case will end with a line containing the name of the station from which Peter needs to travel to all the other stations.

The input finishes with the phantom test case ‘0 0’, which must not be processed.

Output

For every test case, the output will be a line containing an integer, the minimum monthly price that Peter has pay to travel from the given station to all the others, or Impossible if it is not possible to travel to all the stations.

Sample Input

```
3 3
Picadilly
Victoria
Queensway
Picadilly Victoria 2
Queensway Victoria 10
Queensway Picadilly 20
Picadilly
4 2
Picadilly
Victoria
Queensway
Temple
Picadilly Victoria 2
Temple Queensway 100
Temple
0 0
```

Sample Output

```
12
Impossible
```


Problem F. Sum of difference

Time Limit 2000 ms

Problem Statement

Given are N integers $A_1, \dots, A_N$.

Find the sum of $|A_i - A_j|$ over all pairs i, j such that $1 \leq i < j \leq N$.

In other words, find $\sum_{i=1}^{N-1} \sum_{j=i+1}^N |A_i - A_j|$.

Constraints

- $2 \leq N \leq 2 \times 10^5$
- $|A_i| \leq 10^8$
- A_i is an integer.

Input

Input is given from Standard Input in the following format:

```
N
A_1 ... A_N
```

Output

Print the answer.

Sample 1

Input	Output
3 5 1 2	8

We have $|5 - 1| + |5 - 2| + |1 - 2| = 8$.

Sample 2

Input	Output
5 31 41 59 26 53	176

Economic times these days are tough, even in Byteland. To reduce the operating costs, the government of Byteland has decided to optimize the road lighting. Till now every road was illuminated all night long, which costs 1 Bytelandian Dollar per meter and day. To save money, they decided to no longer illuminate every road, but to switch off the road lighting of some streets. To make sure that the inhabitants of Byteland still feel safe, they want to optimize the lighting in such a way, that after darkening some streets at night, there will still be at least one illuminated path from every junction in Byteland to every other junction.

What is the maximum daily amount of money the government of Byteland can save, without making their inhabitants feel unsafe?

Input

The input file contains several test cases. Each test case starts with two numbers m and n , the number of junctions in Byteland and the number of roads in Byteland, respectively. Input is terminated by $m = n = 0$. Otherwise, $1 \leq m \leq 200000$ and $m - 1 \leq n \leq 200000$. Then follow n integer triples x, y, z specifying that there will be a bidirectional road between x and y with length z meters ($0 \leq x, y < m$ and $x \neq y$). The graph specified by each test case is connected. The total length of all roads in each test case is less than 2^{31} .

Output

For each test case print one line containing the maximum daily amount the government can save.

Sample Input

```
7 11
0 1 7
0 3 5
1 2 8
1 3 9
1 4 7
2 4 5
3 4 15
3 5 6
4 5 8
4 6 9
5 6 11
0 0
```

Sample Output

```
51
```

Problem H. Robot Factory

Time Limit 2000 ms

Problem Statement

Takahashi can combine a head part and a body part to create a robot. A robot falls over if the weight of the head part is greater than the weight of the body part.

Currently, he has N head parts and M body parts. The weight of the i -th ($1 \leq i \leq N$) head part is H_i grams, and the weight of the i -th ($1 \leq i \leq M$) body part is B_i grams.

He wants to create a total of K robots that do not fall over by appropriately combining the parts he has. Determine whether he can achieve his goal by combining the parts well.

Here, a part cannot be used to create multiple robots, and two or more head parts (or two or more body parts) cannot be used to create one robot.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $1 \leq K \leq \min\{N, M\}$
- $1 \leq H_i \leq 10^9$ ($1 \leq i \leq N$)
- $1 \leq B_i \leq 10^9$ ($1 \leq i \leq M$)
- All input values are integers.

Input

The input is given from Standard Input in the following format:

```
N M K
H1 H2 ... HN
B1 B2 ... BM
```

Output

Print **Yes** if Takahashi can combine the parts well to create K robots that do not fall over;

otherwise, print **No**.

Sample 1

Input	Output
6 6 3 2 7 1 8 2 8 1 8 2 8 4 5	Yes

If we denote combining the i -th head part and the j -th body part as (i, j) , then Takahashi can create three robots that do not fall over by combining them as $(1, 2)$, $(2, 4)$, $(3, 6)$, for example.

Thus, print **Yes**.

Sample 2

Input	Output
1 1 1 43 1	No

His head part is too heavy, so he cannot create any robot that does not fall over.

Sample 3

Input	Output
1 1 1 100 100	Yes

Note that a robot does not fall over if the head and body have equal weights.

Sample 4

Input	Output
12 15 12 748 169 586 329 972 529 432 519 408 587 138 249 656 114 632 299 984 755 404 772 155 506 832 854 353 465 387	Yes

Problem I. Road Reparation

Time Limit 1000 ms

Mem Limit 131072 kB

There are n cities and m roads between them. Unfortunately, the condition of the roads is so poor that they cannot be used. Your task is to repair some of the roads so that there will be a decent route between any two cities.

For each road, you know its reparation cost, and you should find a solution where the total cost is as small as possible.

Input

The first input line has two integers n and m : the number of cities and roads. The cities are numbered $1, 2, \dots, n$.

Then, there are m lines describing the roads. Each line has three integers a, b and c : there is a road between cities a and b , and its reparation cost is c . All roads are two-way roads.

Every road is between two different cities, and there is at most one road between two cities.

Output

Print one integer: the minimum total reparation cost. However, if there are no solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$
- $1 \leq c \leq 10^9$

Example

Input	Output
5 6 1 2 3 2 3 5 2 4 2 3 4 8 5 1 7 5 4 4	14